IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Date of submission October 28, 2026. Date of publication and date of current version to be assigned by the publisher.

Digital Object Identifier 10.1109/ACCESS.2026.0000000

Adaptive Spill-Buffer Sizing in Lexical Analysis: Evaluating an EWMA–P75 Capacity Policy Against Strong Baselines

HARINITHISH T[1], MOHAMMAD ZAID[1], PAWAN KUMAR[1], and BHUVANESWARI M[1]
[1]School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore, Tamil Nadu, India

Corresponding author: Bhuvaneswari M (e-mail: m.bhuvaneswari@vit.ac.in).

Email addresses: harinithish.2024@vitstudent.ac.in, mohammad.zaid2024@vitstudent.ac.in, pawan.kumar2024@vitstudent.ac.in, m.bhuvaneswari@vit.ac.in

ABSTRACT A lexer must store lexemes of very different lengths, and the policy that sizes its spill buffer decides how often it reallocates, how many bytes it copies and how much memory it wastes. This paper studies an Adaptive Spill-Buffer Sizing policy that sets the buffer capacity from an exponentially weighted moving average (EWMA) and a streaming 75th-percentile (P75) estimate of lexeme length, both updated in $O(1)$ time per token. We compare it with a fixed 16-byte buffer and $1.5\times$ geometric growth. Preliminary single-configuration measurements show that the adaptive policy reduces reallocations by 42.83% and copied bytes by 28.43% relative to the fixed 16-byte baseline, but it performs more reallocations (8,135 versus 4,110) and runs slower (134.1 ms versus 115.0 ms) than geometric growth; its advantage is a lower peak memory (6.2 KB versus 16.0 KB). We therefore position the policy as a memory-efficient rather than a speed-optimal design. Across the tested inputs and configurations, the adaptive policies often reduced reallocations and bytes copied relative to the 16-byte initial-capacity baseline, particularly when the initial buffer was small. These reductions did not translate into a consistent throughput improvement. The results are descriptive of the implementation and environment evaluated and do not establish superiority over the manuscript's exact online-quantile method.

INDEX TERMS Adaptive buffering, compiler design, dynamic memory allocation, lexical analysis, performance evaluation, streaming statistics.

I. INTRODUCTION

Lexical analysis is the first phase of a compiler: it reads a character stream and groups it into lexemes, from which tokens are produced for the parser [**?**], [**?**], [**?**]. Token recognition by finite automata is well understood, and generators such as Lex and Flex are standard tools [**?**], [**?**]. The memory used to hold lexeme text is less studied. Short operators need a few bytes, whereas string literals, comments and long identifiers can exceed a small fixed buffer. When this happens a scanner that copies lexemes must allocate a larger block, copy the live bytes and free the old block.

Two policies are common. A fixed buffer is simple but either reallocates often (if small) or wastes memory (if large). Geometric growth, typically by a factor of 1.5 or 2, gives amortized constant cost per append [**?**], but it ignores the actual lexeme-length distribution and can overshoot. Production scanners such as Flex avoid much of this by scanning inside a large input buffer, so that the token text is a pointer into that buffer [**?**]. An adaptive policy is therefore worthwhile only if it is compared with these stronger alternatives.

This paper makes the following contributions.

1) An EWMA+P75 capacity policy for lexeme buffers with $O(1)$ time and constant state per token (Section **??**).
2) An evaluation against geometric growth on six named corpora (Section **??**).

Further refinements (variance-aware headroom, a scale-adaptive quantile step, copy-free shrinking, class-conditioned estimators and an integer-only datapath) are described in Section **??** as proposed extensions.

II. RELATED WORK

A. LEXICAL ANALYSIS AND COMPILER FRONT ENDS

Standard texts describe lexical analysis and buffering schemes for scanners [**?**], [**?**], [**?**], [**?**]. The classic two-buffer scheme with sentinels [**?**] reads input in blocks so that a lexeme can be addressed in place. Lex introduced the table-driven scanner generator model [**?**], and Flex continues it with an input buffer in which the matched text is addressed in place [**?**]. LLVM illustrates a modern compiler infrastructure [**?**]. Ortin et al. [**?**] empirically compare the Lex/Yacc and ANTLR parser-generator tools in terms of productivity and usability; this supports treating lexer and parser tooling as a subject of empirical evaluation, but the study does not address lexeme buffer capacity. The works above treat the lexeme buffer policy as a fixed design choice.

B. DYNAMIC ARRAY GROWTH

Amortized analysis of geometric array growth is standard [**?**]. It bounds the average cost but does not use the observed data distribution.

C. STREAMING ESTIMATION

EWMA control statistics have a long history [**?**], [**?**]. Streaming quantiles can be tracked with the P^2 algorithm [**?**] or with space-bounded summaries [**?**], and quantiles can be characterised as minimisers of the pinball loss [**?**], which motivates a stochastic-gradient update. Ivkin et al. [**?**] study practical streaming-quantile algorithms and their space and update-time trade-offs, which is relevant to the choice of quantile estimator, although that work is not about lexer buffers. The present work applies a simple version of these ideas to lexeme buffer sizing, building upon established research foundations [**?**].

III. PROPOSED METHOD

A. PROBLEM FORMULATION

Let the t-th completed token have length L_t and let the buffer capacity be S_t. When $L_t > S_t$ the buffer is reallocated to a new capacity, copying B live bytes. The goal is a capacity policy that minimises the number of reallocations A and the copied bytes B without unbounded memory. The hypothesis is that lexeme lengths show temporal locality and skew, so an EWMA and a P75 estimate computed online predict a good capacity for upcoming tokens. This hypothesis is tested in Section **??**.

B. ESTIMATORS

The EWMA of lexeme length is

$$E_t = \alpha L_t + (1-\alpha)E_{t-1}, \qquad \alpha \in (0,1]. \quad (1) eq : ewma$$

Algorithm 1 EWMA+P75 target capacity (per token)

alg:adaptive

Require: L_t, state (E, Q), parameters $\alpha, \eta, C_{\min}, C_{\max}$

$E \leftarrow \alpha L_t + (1-\alpha)E$
if $L_t \le Q$ **then**
  $Q \leftarrow Q - 0.25\,\eta$
else
  $Q \leftarrow Q + 0.75\,\eta$
end if
$R \leftarrow \max(Q, E)$
return $\min\{C_{\max}, \max(C_{\min}, \lceil R \rceil)\}$

The P75 estimate is updated by a pinball-loss stochastic gradient with $\tau = 0.75$ and step η:

$$Q_t = Q_{t-1} + \eta\big(\tau - \mathbb{I}[L_t \le Q_{t-1}]\big). \qquad (2) eq : quantile$$

Thus Q rises by 0.75η when $L_t > Q_{t-1}$ and falls by 0.25η otherwise, and it settles where 75% of lexemes lie at or below Q.

C. CAPACITY POLICY

The target and the bounded capacity are

$$R_t = \max(Q_t, E_t), \qquad (3) eq:target$$
$$C_t = \min\{C_{\max},\ \max(C_{\min}, \lceil R_t \rceil)\}. \qquad (4) eq:capacity$$

When a lexeme of length $L_t > S_t$ arrives, the buffer grows to $\max(L_t, C_t)$ rather than by a fixed factor. Algorithm **??** gives the per-token update.

D. BASELINES

The following policies are compared.

- **Fixed-16**: a 16-byte buffer that grows on demand (the weakest baseline).
- **Geometric 1.5×**: capacity multiplied by 1.5 on overflow.
- **Adaptive EWMA+P75**: the proposed policy.

E. METRICS

Reallocations per 1000 tokens and the relative change are

$$\text{Reallocs/1k} = \frac{1000\,A}{\max(N,1)}, \qquad \Delta(\%) = 100\,\frac{X_{\text{adaptive}} - X_{\text{baseline}}}{\max(|X_{\text{baseline}}|, \epsilon)}. \qquad (5) eq : metrics$$

We also report total copied bytes (raw counts), median runtime and peak memory.

IV. PROPOSED EXTENSIONS (NOT YET EVALUATED)

The following refinements are planned for future iterations.

- **Variance-aware headroom**: add kD_t to the target, where $D_t = \beta|L_t - E_{t-1}| + (1-\beta)D_{t-1}$.

TABLE 1. Bundled evaluation inputs.

tab:corpora

File	Bytes	Tokens	Role
`sample_c.c`	936	189	sample fixture
`stress_long_tokens.c`	1,406	70	stress fixture
`long_identifiers.c`	2,383	136	long identifiers
`long_comments.c`	3,298	62	long comments
`long_strings.c`	2,245	69	long strings
`realworld_c_sample.c`	4,546	682	real-world sample

- **Scale-adaptive quantile step**: $\eta_t = \eta_{\mathrm{rel}} \max(Q_{t-1}, C_{\min})$.
- **Copy-free hysteresis shrink**: shrink to C_t at a token boundary after K consecutive tokens with $C_t \le S/2$.
- **Class-conditioned estimators**: one estimator bank per token class (identifier, number, string, comment), selected from the first character.
- **Integer-only EWMA**: for $\alpha = 2^{-s}$, $\tilde{E}_t = \tilde{E}_{t-1} + ((L_t \ll F) - \tilde{E}_{t-1}) \gg s$, with allocator-aligned capacities.

V. EXPERIMENTAL SETUP

A. IMPLEMENTATION AND ENVIRONMENT

The lexer and estimators are written in C99. The Draft-1 measurements ran on an ASUS Vivobook Pro 15 OLED (AMD Ryzen 7 5800HS, 16 GB RAM) under Ubuntu 22.04 LTS on WSL2, timed with `clock_gettime`. A later benchmark matrix of 6,720 trials (6 datasets $\times$ 7 initial buffer sizes $\times$ 8 policies $\times$ 20 repetitions, i.e., 336 configurations) was run in a Linux container using GCC with flags `-std=c11 -Wall -Wextra -Wpedantic -O2 -Isrc` and `CLOCK_MONOTONIC` timing. All token-stream equivalence checks passed in every trial. CPU model, governor/turbo state, and physical host isolation were not established and remain to be recorded on a native physical Linux machine in future work.

Implementation note. The code that produced the current benchmarked implementation uses a default EWMA weight of $\alpha = 0.20$, estimates P75 from a fixed 4,096-bin histogram, and includes a Markov-conditioned estimate. This differs from the exact algorithm specified in Section **??**, and these preliminary results serve as engineering evidence rather than paper-exact measurements.

B. DATASETS

Six bundled C inputs are used (Table **??**). They combine representative sample code with targeted stress fixtures. Token counts were obtained by running the compiled lexer in compare mode with a 32-byte buffer.

TABLE 2. Experimental parameters

tab:params

Parameter	Symbol	Value
Baseline buffer	C_{fixed}	16 B
EWMA weight	α	0.125
Quantile step	η	0.05
Capacity bounds	$C_{\min}/C_{\max}$	16 B / 4096 B
Corpora	–	6
Trials per configuration	–	20

TABLE 3. Preliminary aggregate measurements from Draft 1.

tab:prelim

Policy	Reallocs	Time (ms)	Peak (KB)
Fixed-16	14,230	120.5	4.0
Geometric 1.5×	4,110	115.0	16.0
Adaptive EWMA+P75	8,135	134.1	6.2

C. PARAMETERS

Table **??** lists the foundational parameters of the proposed method.

VI. RESULTS AND DISCUSSION

A. PRELIMINARY AGGREGATE RESULTS

Table **??** shows the Draft-1 aggregate numbers. Against Fixed-16 the adaptive policy performs 42.83% fewer reallocations and copies 28.43% fewer bytes. Against the stronger geometric baseline, however, it performs more reallocations and is slower, and it wins only on peak memory (6.2 KB versus 16.0 KB).

For a randomized-order rerun, the Max(P75+EWMA) policy was compared with Fixed-16 over the 42 dataset and buffer-size cells with a nonzero baseline. Reallocations were lower in 25 cells, tied in 17 and never higher, with a mean relative change of -42.82%. Copied bytes were lower in 23 cells, tied in 17 and higher in 2 (mean -28.43%). Runtime was higher in 31 cells and lower in 11 (median $+11.34\%$, mean $+21.84\%$). The benefit weakened at large initial buffers, with no benefit at 4,096 and 16,384 bytes, and Fixed-32 and Fixed-64 were competitive on runtime. These figures show that adaptive sizing often reduces reallocations and copied bytes when the initial buffer is small, but that the reductions did not translate into a consistent runtime improvement.

B. RUNTIME OVERHEAD

Draft 1 reported an 11.29% runtime overhead of the adaptive policy over Fixed-16 (134.1 ms versus 120.5 ms). In the cell-level rerun the median runtime change was $+11.34\%$ and the mean $+21.84\%$, with the adaptive policy slower in 31 of 42 cells. A few floating-point operations per token are unlikely to account for this alone. A first `gprof` profile on a repeated long-identifier workload was too coarse to isolate the predictor: the predictor functions rounded to 0.00% of

samples, and the visible samples were dominated by the lexer's consume and next-token paths.

C. THREATS TO VALIDITY

The benchmarks of Draft 1 ran under WSL2, where differences of a few milliseconds are within the noise of the platform. The corpora are all C sources, so results may differ for other languages. Finally, real scanners that address lexemes in place copy lexemes rarely, so the benefit of any copy-reduction policy depends on the scanner design.

This evaluation has several limitations. The bundled corpus set includes targeted stress fixtures and should not be interpreted as a broad sample of independent production codebases. The available implementation differs in estimator details from the algorithm specified in the manuscript; therefore, benchmark results from that implementation are treated as preliminary engineering evidence rather than final measurements of the proposed method. A controlled native-machine rerun, additional baseline implementations, token-level capacity traces, and component-level profiling are required before drawing conclusions about the relative performance of the paper-exact algorithm. Results should be interpreted as allocation/runtime trade-offs rather than evidence of universal throughput improvement.

VII. CONCLUSION

An EWMA+P75 policy reduces reallocations and copied bytes compared with a fixed 16-byte buffer and uses less peak memory than 1.5× geometric growth, but in the preliminary measurements it performs more reallocations and is slower than geometric growth. Across the tested inputs and configurations, the adaptive policies often reduced reallocations and bytes copied relative to the 16-byte initial-capacity baseline, particularly when the initial buffer was small. These reductions did not translate into a consistent throughput improvement: runtime was frequently higher, indicating that estimator and policy overhead can offset allocation savings. The results are descriptive of the implementation and environment evaluated. Future work includes measuring the extensions of Section **??** and a microcontroller deployment.

AUTHOR CONTRIBUTIONS

Harinithish T: algorithm formulation and statistical implementation (EWMA and quantile estimators). Mohammad Zaid: simulation environment, UI dashboard and telemetry backend. Pawan Kumar: C lexer core, memory interfaces, system integration and manuscript drafting. Bhuvaneswari M: supervision and review.

ACKNOWLEDGMENT

The authors thank the School of Computer Science and Engineering at VIT for infrastructure. Generative AI tools were used to assist with drafting, editing and LaTeX formatting of this manuscript; the authors reviewed all content and take full responsibility for it.

FUNDING

No external funding was received.

COMPETING INTERESTS

The authors declare no competing interests.

DATA AND CODE AVAILABILITY

The code, datasets, and benchmark artifacts used in this evaluation are publicly available on GitHub at `https://github.com/Harinithish-T/PLSB-Lexer` (commit hash: `86fdb6f977d4042459109b1284b31dc661e01402`).

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.
[2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 3rd ed. Cambridge, MA, USA: Morgan Kaufmann, 2022.
[3] S. S. Muchnick, *Advanced Compiler Design and Implementation*. San Francisco, CA, USA: Morgan Kaufmann, 1997.
[4] N. Wirth, *Compiler Construction*. Harlow, U.K.: Addison-Wesley, 1996.
[5] M. E. Lesk and E. Schmidt, "Lex—A lexical analyzer generator," Bell Laboratories, Murray Hill, NJ, USA, Comput. Sci. Tech. Rep. 39, 1975.
[6] V. Paxson *et al.*, "Flex: The fast lexical analyzer." [Online]. Available: https://github.com/westes/flex
[7] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
[8] M. Greenwald and S. Khanna, "Space-efficient online computation of quantile summaries," in *Proc. ACM SIGMOD*, 2001, pp. 58–66, doi: 10.1145/375663.375670.
[9] R. Jain and I. Chlamtac, "The P^2 algorithm for dynamic calculation of quantiles and histograms without storing observations," *Commun. ACM*, vol. 28, no. 10, pp. 1076–1085, 1985, doi: 10.1145/4372.4378.
[10] R. Koenker and G. Bassett, "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978, doi: 10.2307/1913643.
[11] S. W. Roberts, "Control chart tests based on geometric moving averages," *Technometrics*, vol. 1, no. 3, pp. 239–250, 1959.
[12] J. S. Hunter, "The exponentially weighted moving average," *J. Quality Technol.*, vol. 18, no. 4, pp. 203–210, 1986.
[13] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis and transformation," in *Proc. Int. Symp. Code Generation and Optimization (CGO)*, 2004, pp. 75–86, doi: 10.1109/CGO.2004.1281665.
[14] F. Ortin, J. Quiroga, O. Rodriguez-Prieto, and M. Garcia, "An empirical evaluation of Lex/Yacc and ANTLR parser generation tools," *PLOS ONE*, vol. 17, no. 3, Art. no. e0264326, 2022, doi: 10.1371/journal.pone.0264326.
[15] N. Ivkin, E. Liberty, K. Lang, Z. Karnin, and V. Braverman, "Streaming quantiles algorithms with small space and update time," *Sensors*, vol. 22, no. 24, Art. no. 9612, 2022, doi: 10.3390/s22249612.
[16] M. Bhuvaneswari, private research communication and notes, 2026.

PLACE PHOTO HERE

HARINITHISH T =
is currently pursuing the B.Tech. degree at the Vellore Institute of Technology (VIT), Vellore, Tamil Nadu, India. Hailing from Vellore, he is currently in his fifth semester and maintains a CGPA of 8.6.

PLACE PHOTO HERE

MOHAMMAD ZAID =
is currently pursuing the B.Tech. degree at the Vellore Institute of Technology (VIT), Vellore, India. Originally from Kolkata, West Bengal, he is passionate about learning new technologies, developing technical skills, and exploring opportunities for personal and professional growth.

PLACE PHOTO HERE

PAWAN KUMAR =
is currently pursuing the B.Tech. degree in Computer Science and Technology at the Vellore Institute of Technology (VIT), Vellore, India. Originally from Sasaram, Bihar, he also works as a freelance full-stack web application developer, specializing in React and Next.js architectures.

PLACE PHOTO HERE

BHUVANESWARI M =
is an Associate Professor with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India.